WEB SECURITY AUDIT REPORT

Passive OWASP ZAP 2.17.0 assessment | Local versus production | 2 November 2026

01 / Executive summary

This report interprets two OWASP ZAP Baseline reports provided by the site owner. No manual exploitation or independent verification was performed. The production scan reported no high-severity alerts. Remaining alerts are indications of configuration or hardening opportunities, not proof of intrusion or exploitable vulnerabilities. Prioritize reviewing the contact endpoint and CORS policy.

Production: 0 high, 4 medium, 2 low, 5 informational alert categories; 41 endpoints. Local: 0 high, 6 medium, 6 low, 2 informational categories; 40 endpoints. These are categories, not counts of confirmed vulnerabilities. ZAP classified 81% of production responses as slow, but this isolated observation is insufficient to establish real-world performance.

02 / Scope and methodology

OWASP ZAP 2.17.0 Baseline scanned a locally served static build and the public domain. Coverage included discovered HTTP responses, headers, links and resources. The local Python server cannot execute the real /api/contact backend and returns HTTP 501. The production scan discovered 41 endpoints (approximately 97% GET and 2% POST). No active exploitation, full code review, authentication tests, load testing or email-delivery validation was carried out.

03 / Findings and safeguards

F01 | Missing anti-CSRF token on contact form | MEDIUM / TO VERIFY

ZAP identified a POST form targeting /api/contact without a hidden CSRF token (two instances). CSRF causes a browser to perform an unwanted action in a user’s context, particularly when automatic session credentials are involved. A public anonymous form without a token is not automatically vulnerable to CSRF; spam, message abuse and resource consumption may be more relevant.

Review backend session/cookie use; validate Origin and/or Referer where appropriate; add CSRF tokens for authenticated flows; apply rate limiting, server-side input validation, a supplementary honeypot and anomaly logging. Verify in staging without sending real messages.

F02 | CSP style-src allows unsafe-inline | MEDIUM / CONFIGURATION CONFIRMED

The production policy includes style-src 'self' 'unsafe-inline' https://fonts.googleapis.com. CSP restricts which resources a browser may load; unsafe-inline permits embedded CSS and weakens style-injection protection. This does not establish a confirmed XSS vulnerability. Script hashes and script-src-attr none are positive existing controls.

Inventory inline styles; move them to external stylesheets or use appropriate hashes/nonces; test first with Content-Security-Policy-Report-Only; retain script restrictions and frame-ancestors none. Check browser console and visual regressions.

F03 | Wildcard CORS origin | MEDIUM / CONTEXT DEPENDENT

ZAP observed Access-Control-Allow-Origin: * on public HTML, CSS, robots.txt and sitemap.xml. CORS determines whether JavaScript from another origin can read browser responses. A wildcard is often acceptable for intentionally public resources; it does not prove private-data exposure. API responses require separate scrutiny.

Review Cloudflare/Workers rules per route; remove unnecessary CORS; restrict allowed origins on endpoints that need it; do not expose sensitive API responses; check preflight and error responses; avoid combining arbitrary origins with credentials.

F04 | Missing SRI on external stylesheets | MEDIUM / SUPPLY-CHAIN HARDENING

Google Fonts stylesheet links lack integrity attributes. Subresource Integrity (SRI) verifies external content against an expected hash. Google Fonts CSS can vary by browser, so a fixed hash may break font loading. This finding is not evidence of compromise.

Prefer self-hosting required fonts and CSS; reduce font families; use integrity and crossorigin for stable, versioned third-party assets. Test rendering and compatibility after changes.

F05 | Missing COEP | LOW / OPTIONAL

Cross-Origin-Embedder-Policy is absent on three HTML responses. COEP, alongside COOP, can enable cross-origin isolation. Not every website requires it, and require-corp may block external fonts or resources.

Determine whether isolation is needed; test third-party dependencies; enable require-corp or credentialless only with a justified use case and regression testing.

F06 | HSTS missing on two API responses | LOW / VERIFY PER ROUTE

Strict-Transport-Security is absent on /api/ and the /api/contact response, while the main page sends HSTS. HSTS tells browsers to prefer HTTPS. Missing it on individual paths does not mean the site is serving insecure traffic.

Check Cloudflare/Workers and error responses; apply a consistent HTTPS/HSTS policy where appropriate; verify HTTP-to-HTTPS redirects and includeSubDomains implications before changes.

04 / Informational alerts and local comparison

The five informational categories cover a benign comment in contact-confirmation.js (the word “from” triggered the rule), non-storable content, Cache-Control revalidation, cacheable public assets, and responses requiring revalidation. These do not independently demonstrate a security incident. The local Python server does not reproduce Cloudflare headers or Workers; missing clickjacking protection, nosniff, permissions headers and Python version disclosure in the local scan must not be attributed to production. ZAP reported zero alerts manually marked as false positives; this does not prove every alert is applicable.

05 / Prioritized remediation

P1 (1–3 days): review /api/contact input validation, Origin, rate limiting, anti-spam and session dependencies; audit CORS rules on API and error routes. P2 (1–2 weeks): reduce/self-host Google Fonts; progressively remove unsafe-inline after Report-Only testing; verify HSTS consistency. P3 (as needed):

decide whether COEP is beneficial. Ongoing: rerun ZAP Baseline after changes, store dated evidence, monitor Core Web Vitals (LCP, INP, CLS), maintain dependency updates, keep secrets out of the repository, and record residual-risk decisions.

06 / Technical glossary

Endpoint: an HTTP route such as /api/contact. HTTP 2xx/3xx/4xx/5xx: success/redirect/client error/server error. GET retrieves data; POST typically submits data. HTTP header: response metadata. CSP: resource-loading policy. CSRF: cross-site request forgery. CORS: browser cross-origin read permissions. SRI: integrity hash for external assets. HSTS: browser HTTPS enforcement. COEP/COOP/CORP: cross-origin isolation and resource policies. X-Frame-Options/frame-ancestors: clickjacking protections. X-Content-Type-Options: nosniff prevents MIME sniffing. Cache-Control: response caching rules. False positive: an alert not applicable after validation. Residual risk: risk remaining after safeguards.

07 / Evidence, references and limitations

Primary evidence: OWASP ZAP Baseline local and production reports dated 2 November 2026, plus production HTTP headers supplied by the owner. Technical references: OWASP ZAP Baseline Scan; OWASP CSRF Prevention Cheat Sheet; OWASP CSP Cheat Sheet; MDN CORS, SRI, HSTS and COEP; OWASP Secure Headers. High confidence that quoted headers appear in supplied evidence; moderate confidence in impact without backend code, Cloudflare configuration, functional tests and logs. This is a passive automated assessment interpretation, not a security certification or comprehensive penetration test.

https://www.zaproxy.org/docs/docker/baseline-scan/

https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html

https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS

https://developer.mozilla.org/en-US/docs/Web/Security/Defenses/Subresource_Integrity

https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Strict-Transport-Security

https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Cross-Origin-Embedder-Policy

https://owasp.org/www-project-secure-headers/